



MERRIMACK COLLEGE

CSC 6303

Week 1

Programming languages classification and families

Systems and Languages Survey - Dr. Paulo Fernandes

Presentation Agenda

Week 1

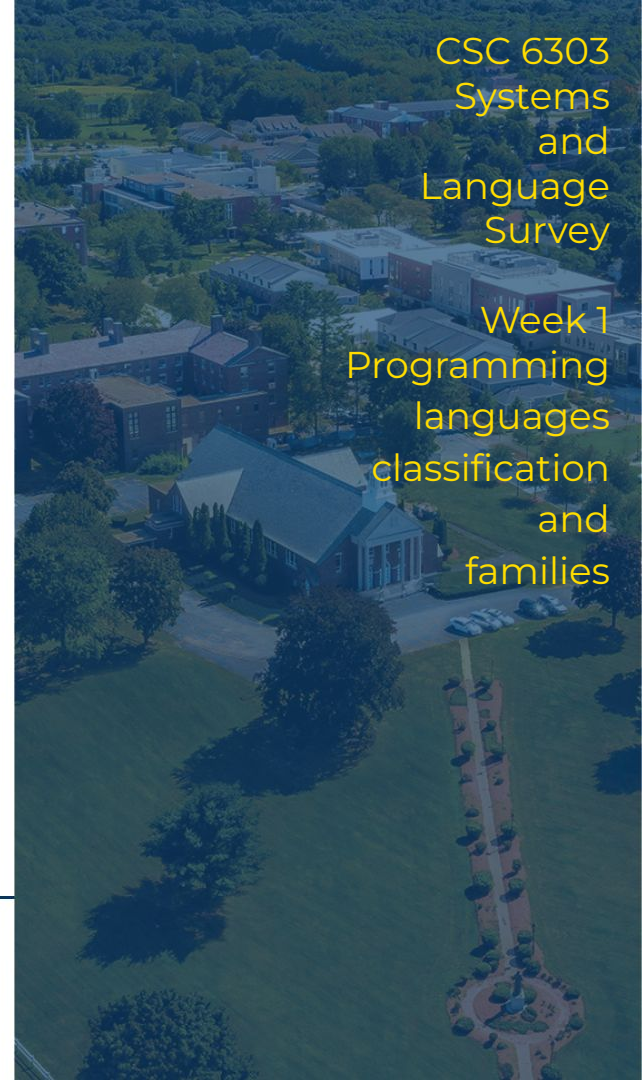
- Course Introduction
 - a. Course format
 - b. Evaluation
 - c. Timeline
- Programming languages families
 - a. The procedural languages
 - i. according to execution
 - ii. according to object orientation
 - b. The non-procedural languages
- This Week's tasks



MERRIMACK COLLEGE

CSC 6303
Systems
and
Language
Survey

Week 1
Programming
languages
classification
and
families



Course Introduction

This and the next seven weeks ahead

Welcome!

This course is entitled CSC 6303
Systems & Languages Survey

This is the third of four advanced
courses in the program and it is
intended to introduce you to some
of the more popular languages
and operating systems in use.

- Course format
- Timeline
- Evaluation



Course Format

Success Coach and Graduate Advising

- ecs-grad-advising@merrimack.edu

This course is fully online, all weeks will have:

- Live sessions every Wednesdays 6:30pm to 8:30pm
- Office Hours every next Tuesdays 8:30pm to 9:30pm

Some weeks will have:

- Discussion/In-class exercises tasks until Saturday
- Quizzes to be taken from Saturday to Next Wednesday
- Coding assignments to be turned in until next Wednesday

Last week has:

- Final exam to be done until last Saturday

Visit the Hub!

Send me an email if you are planning to attend office hours

Email subjects have to start with CSC6303

Communication

- fernandesf@merrimack.edu

Zoom chat per request



MERRIMACK COLLEGE

Course Format

- Live sessions every Wednesday 6:30pm to 8:30pm
 - Meeting ID: 813 6523 4335 Passcode: CSC6303
- Office Hours every Friday 6:30pm to 7:30pm
 - Meeting ID: 813 6523 4335 Passcode: CSC6303
- In-class exercises, a.k.a. discussion tasks until Saturday
 - Simple posts on the discussion board about the topics of the week
- Quizzes to be taken from Saturday to Next Wednesday
 - Ten multiple choice questions, open book, open notes
- Coding assignments to be turned in until next Wednesday
 - Programming tasks
- Final exam to be done until last Saturday of classes
 - Eight questions, 4 hours to complete after starting, open book, open notes.



Timeline

This is a very fast-paced course (as usual).

Try to check it out the topics before the live session.

Any change will be informed in class and on Canvas.

Unit	Week	Topic	Project	Quiz	Exam
1	1	Programming languages classification and families	Project #1		
2	2	From Python to Java and C/C++	Project #2	Quiz #1	
2	3	From Python to Go, Swift, and modern languages	Project #3		
2	4	From Python to JavaScript and Perl	Project #4	Quiz #2	
3	5	Operating systems basic concepts and organization	Project #5		
3	6	File systems basic concepts and styles	Project #6		
4	7	Operating systems examples: Linux, MacOS, and Windows	Project #7	Quiz #3	
5	8	Multi-thread, parallelisation, and scheduling			Final Exam (all units)



MERRIMACK COLLEGE

Don't let tasks pile up!

Evaluation

The evaluated tasks are weighted as such (105%):

Activity	Points	Percentage
Projects (7)	210 (30, 30, 30, 30, 30, 30, 30)	52.5%
Quizzes (3)	75 (25, 25, 25)	18.75%
In-Class Exercises (7)	35 (5, 5, 5, 5, 5, 5, 5)	8.75%
Final Exam	100	25.0%

All tasks are due in their stated deadline, the late penalties reduced 10% of the evaluation, plus 2% for each full day of delay.



MERRIMACK COLLEGE

Final deadline on Friday of the course's last week

Evaluation

The final letter grade is computed according to this table:

A	A-	B+	B	B-	C+	C	C-	F
95 and up	90 to 94.9	87 to 89.9	83 to 86.9	80 to 82.9	77 to 79.9	73 to 76.9	70 to 72.9	69.9 and low

There are no grade D+, D, or D- in graduate courses.

An average grade B is required to the Masters of Science in Computer Science, lower than that puts you in probation.



MERRIMACK COLLEGE

Final grades published first Monday after the course's end.

Programming languages families

The purpose and expressiveness are always the same, the format is not.

All programming languages derived from the Turing machine definition: tasks executed according to a form of memory, tasks include input and output operations, and decisions taken based on input and memory. However, there are several ways to do it.

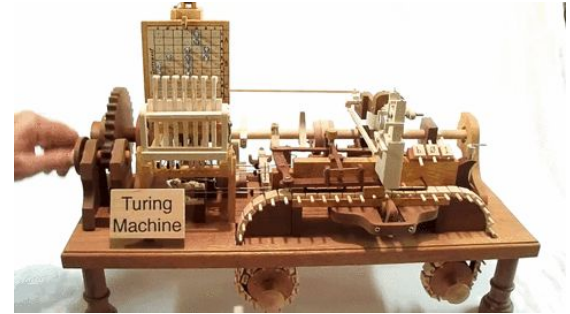
- The procedural languages
 - execution format
 - object-oriented approaches
- The non-procedural languages



Programming Language Families

A **Turing Machine** is composed of a tape with symbols that can be read or written upon.

Commands also include moving the tape back and forth, but most of all, the Alan Turing's universal machine innovation was the capacity to read the tape and decide which command to execute (1920's).



It is formally proven that a Turing machine is capable to execute anything that any modern programming language is capable of (and vice-versa).

Later on, the idea of a memory-stored program (commands) was introduced by John Von Neumann in the 1940's (**Von-Neumann Architecture**).

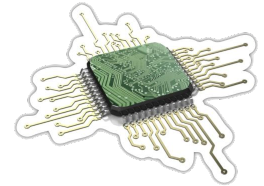


MERRIMACK COLLEGE

All programming languages have the same expressiveness, and, as of now, all follow the Von-Neumann architecture.

In the beginning ... and still

- All commands in a computer translate into an executable code in binary.
- Each processor has a set of commands encoded in binary, as memory addresses are also encoded in binary, including input and output peripheral access (spoolers, serials, etc).
- This form of program, binary code, is called machine language (or machine code) and it is specific to every machine (chipset).
- In the early days of modern computing the programs was hand written as such by humans, but soon enough simple programs were written to pass from a human simple, but readable, language called assembly language to the machine language.



MERRIMACK COLLEGE

Text: [Definition of Machine Language](#)

Video: [Assembly Language in 100 Seconds](#)

Procedural Languages - Algol

Algol

A procedural language is a language that defines commands that are sequentially executed, a list of tasks to be executed following a control flow.

- Most programming languages follow this approach and are, therefore, procedural languages.

Assembly languages are procedural languages.

Initial high level languages, i.e., languages more abstract than assembly languages, were all procedural languages, as for example Algol language.

```
begin
  real x;
  integer i, j;
  for i := 2 until Z do begin
    x := Y[i];
    for j := i-1 step -1 until 1 do
      if x >= A[j] then begin
        A[j+1] := x; goto Found
      end else
        A[j+1] := A[j];
    A[1] := x;
    Found:
  end
end
end Sort
```

Bubblesort in Algol ([source](#)).



The children of Algol

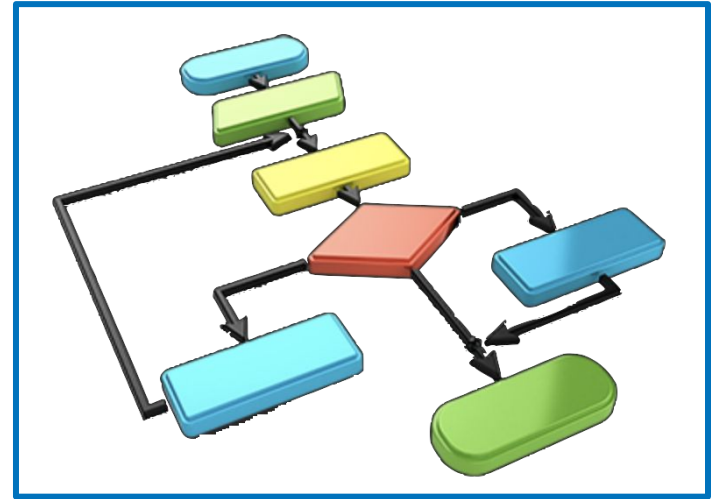
Other procedural languages became popular by specific usage:

- COBOL for database manipulation;
- FORTRAN for scientific (mostly physics) processing;
- BASIC for occasional coding.

ALGOL was developed having programming paradigms in mind, not applications, as such it was the inspiration for modern procedural languages as:

- PASCAL and ADA (very close to ALGOL);
- C (an ergonomic version of ALGOL ideas).

ALGOL - Algorithmic Language



C Language

Dennis Ritchie 1972



MERRIMACK COLLEGE

Text: [Programming History: The Influence of Algol \(pt1\)](#) [\(pt2\)](#)

The C language

While working at Bell Labs, Dennis Ritchie create some prototyped languages, the third attempt, after languages A and B, was called C. It followed the principles of imperative procedural programming, just as ALGOL, but it was ergonomic:



- Short reserved words:
 - **int** instead of **Integer**;
- Compact commands:
 - **for (i=0; i<10; i++)** for a *for* loop of $i=0$ to 9
 - **a=(b<c)?b:c** for a being assigned the smallest value between b and c ;
- Efficiency over clarity:
 - **&a** to refer to the address of where the variable a is stored.



The children of C

The **C** language was quickly adopted by both industry and academy, even though it was better suited for close to hardware application, such as embedded systems (which still is the most common use for C language).

Practical high-level application industry, however, found **C** not suited for the task due to:

- lack of clarity, leading to high development costs;
- absence of redundancy in the code, leading to sloppy code.

Because of it, many evolutions of **C** were proposed:

- **C++** in 1985 introducing object-oriented concepts and some easier definitions, but keeping C compatibility;
- **Java** in 1995 going deeper into the object-oriented concepts and focusing on industry standards as software reuse.



The grandchildren of C

The popularity of C++ mostly in academy, and the large adoption of Sun Systems' Java in the industry motivated several further programming languages:

- **C#** by Microsoft aimed to provide embedded checking (type, bounds, initialization), internationalization, and most of all integration with the platform .NET;
- **Python** originally by Guido van Rossum, then Python Software Foundation, idealized as an easy to code interpreted language;
- **JavaScript** (JS) is a language as old as Java, and despite the name, unrelated to Java, created for web scripting;
- **Go** by Google is a general purpose language intended to be a programming language on its own, but also portable to other languages (cross-compilers);
- **Swift** by Apple is a language designed to develop applications to Apple's apps to mobile and stand alone device alike.



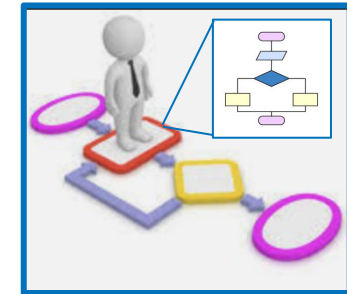
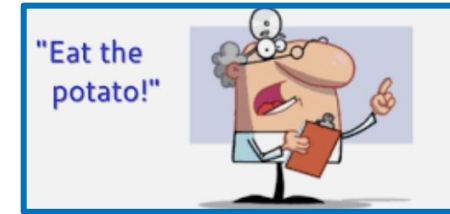
Procedural Languages

Languages based on high-level description of series of mandatory commands reflecting sets of machine language commands are called **imperative languages**.

Such languages have a control flow (stored program) that dictates what is going to be executed next. When commands can be encapsulated into modules imperative languages are called procedural languages (a.k.a. modular languages).

Most algorithms, that were usually described with flow charts, are directly mapped into procedural languages.

All languages seen so far are procedural languages.



Execution Format: Compiled × Interpreted

Performance

Some procedural languages are **compiled**, i.e., the high-level code is converted into machine language (by a compiler, possibly linking several compiled modules).

The executable code (machine language code) is then executed over the operating system.

This is the standard procedure for traditional procedural languages like C and C++.

All code can be compiled or have an interpreter build for it.

Flexibility

Some procedural languages are **interpreted**, i.e., the high-level code is executed by an interpreter.

The interpreter is a compiled program (usually in a different language) that decodes the interpreted language commands into executable tasks.

This is the standard procedure for script procedural languages like Python and JavaScript.



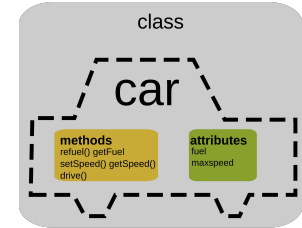
MERRIMACK COLLEGE

Text and Video: [Interpreted vs. Compiled Languages](#).

Object-Oriented Programming Paradigm

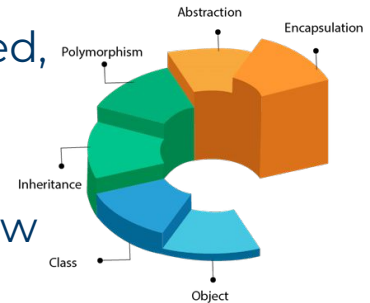
Object-Oriented (OO) Programming is the dominant programming paradigm.

It is a subdivision of the procedural programming, since it remains procedural but center the code description in coding for modules for specific user or system defined data structures. The ensemble defined data structures and methods is called object.

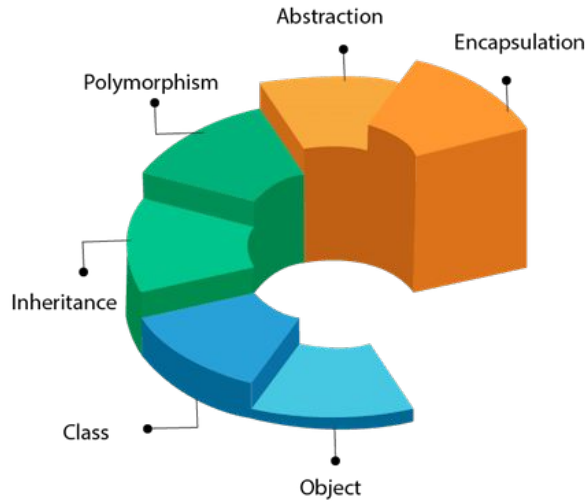


While vanilla procedural programming, as in C, is based on building a program that call modules that have variables defined, in OO paradigm a program is a set of interacting objects.

All modern high-level languages provide some form of OO paradigm features, but the programmer usually may decide how much of OO concepts will be employed.



Object-Oriented Programming Paradigm



In **C** there is no OO concepts, even though you can define user defined data structures.

In **C++** or **Python**, for example, your program or parts of it can be a procedure (and imperative module) that may create and manipulate objects (call methods for this object).

In **Java** everything, even your whole program, is an object and starting the program corresponds to create the object and usually call the method "*run*".



The non-procedural languages - Functional

Although procedural languages largely dominate the scene, some other programming paradigms have been used to specific applications:

- **Functional Languages** can be seen as a extreme approach to modular programming as every definition in a program is expressed as a function, i.e., a module that receives an input and delivers an output. This approach is similar to the classic math definition where every operation can be defined as a mathematical function $f(x) \rightarrow y$ where a function f receiving a set of parameters x is associated to a set of output parameters y .
- The more common languages dedicated to functional programming are **Haskell** and **LISP**, but the concept of functional programming can also be applied to traditional procedural languages as well.

Video: [Functional Programming & Haskell.](#)



MERRIMACK COLLEGE

Text: [Learn Haskell in 10 minutes.](#)

The non-procedural languages - Logic

Although procedural languages largely dominate the scene, some other programming paradigms have been used to specific applications:

- **Logic Languages** can be seen as a mathematical logic expression based concept. A program is defined as a series of facts and logic propositions. The program execution is actually an application of inference rules over the defined facts and logic propositions. While procedural languages are imperative, logic languages are declarative.
- The most common language dedicated to logic programming is **Prolog (Datalog)**, which is a programming language that have been researched and applied mostly in the 90's. However, new the concepts in Computer Science, as Quantum computing, may bring logic languages into the spotlight in the future.



This Week's tasks

- Post discussion
- Programming project #1

Tasks

- Tell about the computer languages you have already use and summarize the difference between them.
- Convert a simple Python code into any other programming language.



Post Discussion - Week 1 - D#1

Tell about the computer languages you have already use and summarize the difference between them.

- You know Python for sure, then describe the differences between Python and some of the other languages you know.
 - For example, are they procedural? They are are easy code?, Did it took long to learn it?, What is your level of expertise in each language?, etc.
 - In the rare case you know only Python pick another language and try to learn the basics before you post your answer.

Your task:

- Post your discussion in the message board until this Saturday;
- Reply posts of your colleagues in the message board until next Wednesday.



MERRIMACK COLLEGE

This task counts towards the Discussions grade.

Progr. Project #1 - Week 1 - P#1

- The following Python code needs to be converted to the programming language of your choice:

```
1  # Python program to find the factorial of a number provided by the user.
2  def fact():
3      n = int(input("Enter an positive Integer: "))
4      while ((n < 0)):
5          n = int(input("Sorry, only positive numbers, enter again: "))
6      if (n == 0):
7          print("The factorial of 0 is 1")
8      else:
9          f = 1
10         for i in range(1,n + 1):
11             f *= i
12         print("The factorial of",n,"is",f)
13
14  fact()
```

The input/output operations may be freely adapted according to the language you choose.



Progr. Project #1 - Week 1 - P#1

- Pick a language that you know already (or one that you will learn the basics now) and convert the Python code in the previous slide into this language.
 - The program in the language of your choice must run exactly as the Python one;
 - Do not hesitate to contact me, preferably in the Office Hours, if you have any question;
 - Make sure to include in the remarks of your code your name, the language and IDE/compiler you tested your generated code.

Your task:

- Save your program code and submit it in the appropriate Canvas until Next Wednesday.

This assignment counts towards
the Coding Projects grade.



MERRIMACK COLLEGE

” Welcome to CSC 6303

- Read the syllabus
- Post in the discussions until Saturday, reply until next Wednesday
- Do the coding project until next Wednesday

Next Week - From Python to Java and C/C++



MERRIMACK COLLEGE